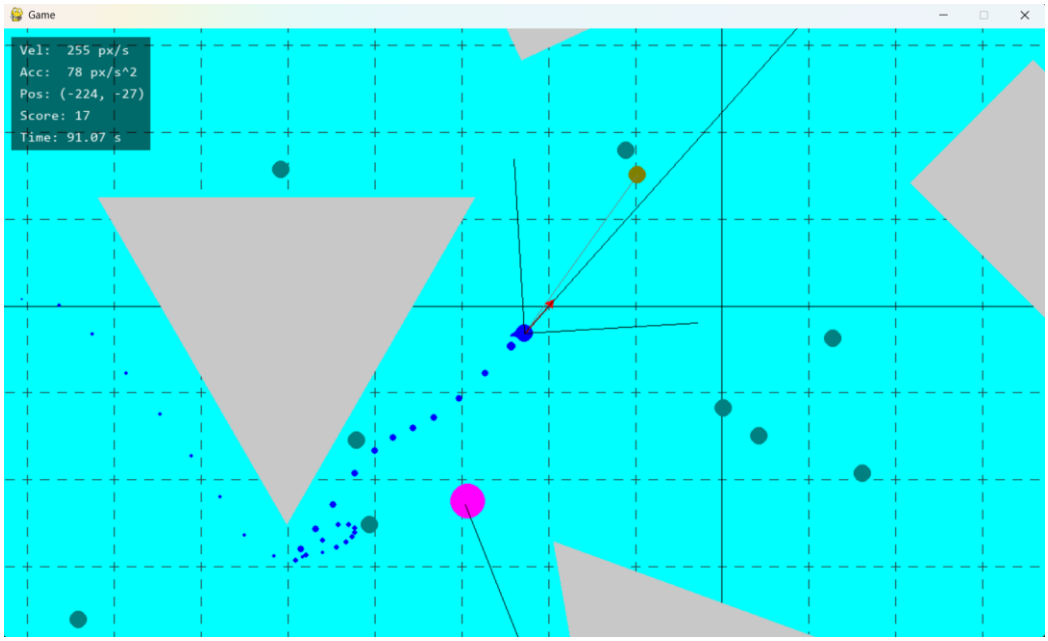
Assignment 1

Name: LI-AN, LO || Student number: N96144323

Github Link: https://github.com/Wattgo-Real/AIForGame_Assignment_1

This game is designed as an open world, with the background consisting of a grid made of solid and dashed lines. The distance between dashed lines is 100 px, while the distance between solid lines is 1000 px.



1. System Design

In this game, I use PyQt5 to create the launch interface. The interface provides multiple options that can be enabled or disabled to control agents, obstacles, and whether the game can be player controlled.

- **The game includes the following elements:**

  a. **Target (Teal color dot)**
  b. **Swamp (Olive color dots)**
  c. **Obstacle (Light gray color surface)**
  d. **Main Agent (Blue color dot)**

     The main agent approaches the target using the seek behavior,

while also fleeing from swamps and avoiding obstacles.

* If main agent reaches target 100 times, this game will end.

e. **Attendant Agent (Purple color dot)**

The attendant agent predicts the main agent's position 0.1 seconds ahead and follows it, but tries not to get too close.

- **The game includes the following toggle controls.**

a. **KeyBoardControl (The initial setting is false. Switch: Key Z)**

If True, the target position can be controlled by arrow keys, and the camera will follow the target. If False, the camera will follow the Main Agent.

b. **MakeObstacle (The initial setting is true. Switch: Key X)**

If True, obstacles will be added into the game.

c. **AgentObstacleDetection (The initial setting is true. Switch: Key C)**

If True, the agent will detect the obstacle and try to avoid it. If False, the agent will ignore the obstacle and may collide with it.

d. **AddAttendantAgent (The initial setting is true. Switch: Key V)**

If True, an Attendant Agent will be added into the game, which will pursue the Main Agent and evade if too close to it.

- **This game will run at 60 fps. And screen width and height are 1200, 700**

- **Update the loop as follows:**

a. **Draw background, colliders, target, swamp and agents.**

Reference Function:

```
self.DrawBackground()
self.DrawObstacle()
self.DrawTargetAndSwamp()
self.DrawAgent()
```

b. **Handles the agent movement, and collision detection.**

Reference Function:

```
self.AgentController.AgentMovement(...)
self.AgentController.Update(...)
```

self.AgentController.ObstacleCollision(...)

c. **Draw debug items. (collision, direction, velocity arrow, acceleration arrow)**

   Reference Function:

   self.MultiAgentCollisionDebugging(...)

   self.AgentStateDraw(...)

   self.CheckTarget(...)

   self.DrawDebugRayWithWhiskers()

d. **Handles keyboard input, camera position and draw HUD(Head-Up Display).**

   Reference Function:

   self.KeyBoardDetectionAndSetCamera()

   self.DrawHUD(...)

2. Behavior Design

A. Adjusting agent's steering. Calculate the acceleration of agents according to the target, swamp, and obstacles, and set the acceleration to agents.

I. **About main agent:**

   Input: Agent, Target, Swamps, Obstacles

   Output: Agent's acceleration

   Key Parameters:

   flee_steer_acc -> flee from swamp

   When the target is generated too close to a swamp, the agent may be unable to reach it. To avoid this issue, the system is designed such that when the agent's speed decreases, the swamp detection range is also reduced. This allows the agent to approach the swamp more slowly and eventually reach the target.

   seek_steer_acc -> seek to target

   If the agent is too close to the swamp, it will stop seeking the target. And limit the maximum seek acceleration when the agent is

far from the target.

obs_steer_acc -> use ray with whiskers to detect obstacles and avoid it.

The steering direction and magnitude are determined based on the collision conditions of the three rays, with the following conditions used to adjust the final steering.

a. When the front ray predicts that the main agent will collide with an obstacle within the next 0.5 seconds, a flee behavior is applied to move the agent away from the obstacle.
b. When **both whisker rays detect an obstacle**, the steering direction is **defined** to be **aligned with the velocity direction**. Additionally, if the angle between a whisker ray and the obstacle plane is too small, no steering is applied (see the Discussion section).
c. If **all rays detect an obstacle**, the agent will steer according to the closer one whisker.
d. When **front ray** and **only one of the whiskers** rays detects an obstacle. ->
   If the two hit points are close and have the same normal, it means that the front ray and the whisker are detecting the same obstacle edge. So, the agent will steer according to the front ray to avoid oscillation. Otherwise, it will steer according to the whisker.
e. If the front ray does not detect an obstacle, but the whisker detects an obstacle, the agent will steer according to the whisker

Blending Behavior:

If the agent is about to collide with a wall, set seek_steer_acc and flee_steer_acc to 0 (Focus on not bumping into the wall), and final output steer will be:

(flee_steer_acc + seek_steer_acc + obs_steer_acc * 3) * acc_scale

II. **About attendant agent:**

Input: Agent

Output: Agent's acceleration

a. Predict the main agent's position 0.1 seconds as new position.

b. If the agent is too close to the main agent's new position, it will flee from the main agent. Otherwise, it will seek to the new position.

B. Inter-Agents' collision:

Agents are assigned mass (radius^2), and two-body collision formulas are used to determine their velocities and directions after collisions.

C. Obstacle:

After updating the agents' positions, the system detects collisions with obstacles and reflects the agents' velocity using the normal of the collision surface.

3. Experimental Setup

This experiment begins with the basic **seek** and **flee** behaviors (main agent), then incorporates the **pursue / evade** behavior (attendant agent) and inter-agent collision, finally evaluates collision handling between agents and obstacles.

A. Seek and Flee Test

The purpose is to test the main agent's Seek and Flee behavior.

• In this sitting

```
        self.KeyBoardControl = True
        self.MakeObstacle = False
        self.AgentObstacleDetection = False
        self.AddAttendantAgent = False
```

• Main agent will seek to target and flee to target by steering, if main agent reaches the target, it gets one point.

• Main agent spawn in (0,0), max speed set to 500 px/s, max acceleration set to 999 px/s^2

• Target spawn function (only one target to the game):

```
target = np.random.randn(2) * 500
```

• Swamp spawn function (every time main agent reaches the target, add swamp to the game)

```
Swamp_list.append(np.random.randn(2) * 500)
```

B. Pursue and Evade Test

The main purpose is to test the attendant agent's behavior in response to the main agent's pursue and evade actions.

• In this sitting

```
        self.KeyBoardControl = False
        self.MakeObstacle = False
        self.AgentObstacleDetection = False
        self.AddAttendantAgent = True
```

• Compared to part A, an attendant agent adds 100px to the right of the main agent. To prevent the main agent and the attendant agent from overlapping, enabled Inter-Agents' collision.

• The attendant agent predicts the main agent's position 0.1 seconds ahead and follows it, but tries not to get too close

C. Obstacles Test

Collisions exist between obstacles and the agent, and the main agent will actively avoid obstacles. The main purpose is to study how to design the main agent so that it does not get stuck against walls while still approaching the target.

• In this sitting

```
        self.KeyBoardControl = False
        self.MakeObstacle = True
        self.AgentObstacleDetection = True
        self.AddAttendantAgent = True
```

• Four obstacles are added to this game. (Two rectangles and two triangles.)

• If the agent hit the obstacles, it will bounce off.

• Main agent use ray with two whiskers to avoid obstacles.

4. Analysis:

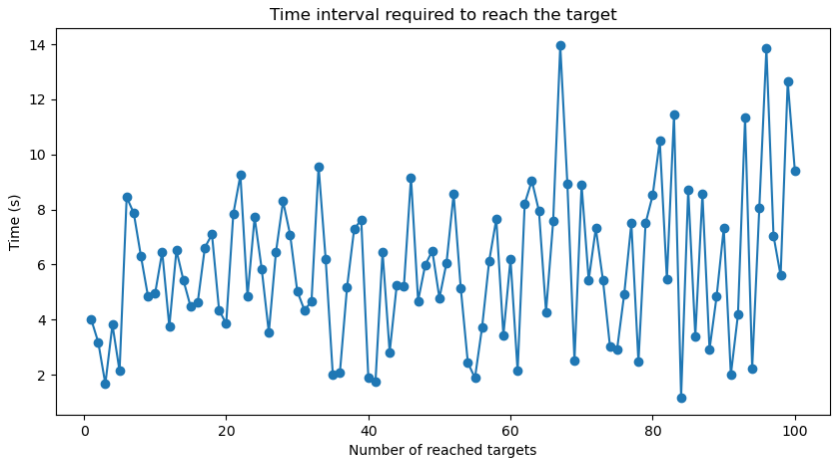
All the results based on np.random.seed(42)

A. Comparison of time to reach 100 points and inter-agent collision count, with vs. without obstacles collision.

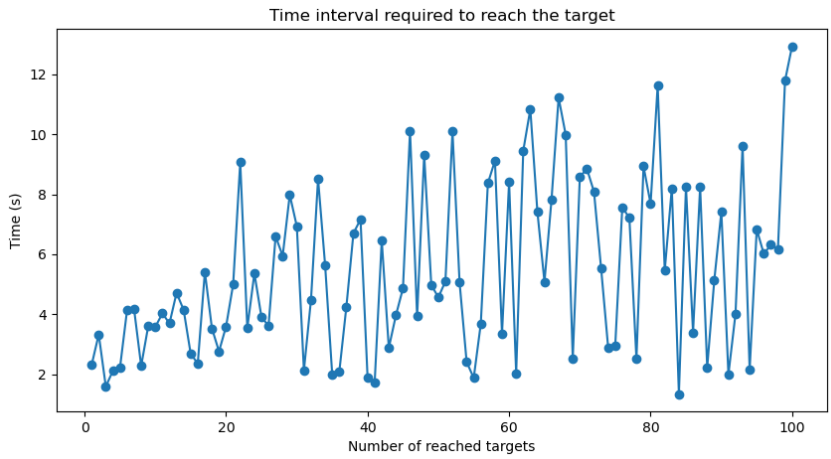
	Total time spent	Collision count
With obstacles collision	590 s	5
Without obstacles collision	546 s	8

The following is a figure showing the time interval required to reach the target. As the number of swamps increases, the time required for the main agent also becomes longer.

I. With obstacles collision.



II. Without obstacles collision.



From the results, it can be observed that introducing collision bodies not only increases the time required for the main agent to reach the target, but also leads to more frequent collisions between agents. This is likely because, in the presence of obstacles, the main agent's velocity is constrained, causing the attendant agents to have insufficient time to avoid collisions.

Additionally, as the number of swamps increases, the time required for the main agent to reach the target also becomes longer.

B. Comparison of Keyboard Target Control (kinematic) and Main Agent (steering).

The Keyboard Target Control is not controlled using acceleration, but only velocity. As a result, when collisions occur, it cannot properly produce the expected bounce behavior. This is also why I do not assign collision to the target. In contrast, the steering implementation of the main agent can handle collisions more easily.

C. Static obstacle collision and moving-agent collision.

When handling collisions with static obstacles, it is sufficient to consider the normal of the surface being hit, which makes collision handling relatively straightforward. However, collisions between moving agents require consideration of multiple factors, such as the other agent's velocity, mass, and angular velocity. This is also why I designed the agents as circles, to reduce the complexity of these considerations.

5. Discussion:

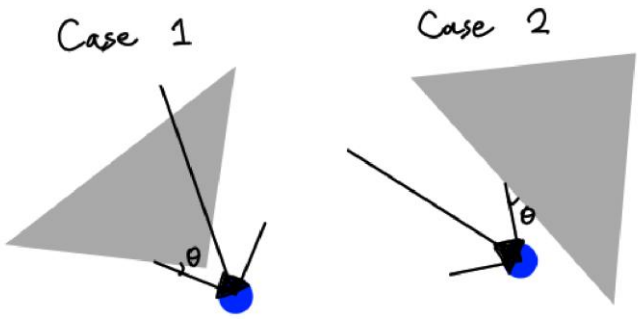
Strengths ->

1. **For obstacle detection, since steering is determined based on the velocity direction, the main agent is unlikely to get stuck on obstacles.**
2. **The main agent successfully completed 100 target-reaching tasks (each time the target is reached, the number of swamps increases).**

Limitations or failure cases ->

1. (Issue fixed) In obstacle avoidance, when the main agent approaches a corner of a rectangle on a plane, it may exhibit oscillation behavior. This occurs because the whiskers on both sides conflict with each other: the left whisker attempts to steer the agent to the left, but in the next frame, the right whisker instead attempts to steer it to the right.

   To fix this problem, I observed that in certain cases, the angle between the ray and the collision surface becomes very small. This may occur when the agent is moving away from the surface or when it is about to collide with it, as shown in the figure below.

   Therefore, in the steering logic, when a ray satisfies the above condition, no steering force is applied to the agent. This effectively prevents the agent from getting stuck at the corner of a rectangle.

2. In obstacle avoidance, when approaching a corner of a rectangle, if the direction of travel is parallel to one of the faces on either side of the corner, the agent may get stuck.

3. The attendant agent may get stuck at corner because the computed collision normals are always based on the edges of the obstacle, without considering its vertices. As a result, when the attendant agent collides with the corner of an obstacle, an incorrect reflection direction may occur.